Bayesian estimation I

This version: October 29, 2026

ECMA 31150, Spring 2026

University of Chicago

Department of Economics

Mikkel Plagborg-Moller

Plan for this slide deck

1. Bayesian inference: using prior knowledge to inform estimation
2. Computational techniques
3. Bayesian software: Stan

Frequentist vs. Bayesian inference

- So far in this course we have conducted **frequentist inference**: we sought statistical procedures that worked well *across repeated (hypothetical) histories of the world*, regardless of what the true model parameters are.
- **Bayesian inference** instead prioritizes some parameter values over others and seeks optimal statistical procedures given the *actually observed history of the world*.
- Before the data analysis, a Bayesian decides which parameter values to prioritize by selecting a **prior distribution** over the param. space.
- The prior can be used to incorporate additional scientific knowledge that the researcher possesses before seeing the data.
  - This can be used to **shrink** the frequentist's parameter estimates toward values that are *a priori* more "reasonable".

Bayesian inference

- One could teach a whole course on Bayesian statistics.
- Bayesian methods have deep methodological underpinnings that can be formalized based on decision theory (essentially, the same stuff you learn in Econ Micro Theory 101).
- However, I will just tell you the bare minimum required in order to productively use the Bayesian software "Stan".
- For theoretical details, see the textbooks on the reading list:
  - Herbst & Schorfheide
  - Gelman et al.

Bayes' rule

- Let θ be a parameter and $f(\theta)$ the prior density over the parameter space. High values of $f(\theta)$ are given to "more likely" θ.
- Let $\boldsymbol{Y}$ be the observed data. Its density, given our model with parameter θ, is $f(\boldsymbol{Y} \mid \theta)$. This is called the **likelihood function**.
- **Bayes' rule**: after we see the data, our updated belief about θ is given by the **posterior** density

$$f(\theta \mid \boldsymbol{Y}) = \frac{f(\boldsymbol{Y} \mid \theta) f(\theta)}{f(\boldsymbol{Y})},$$

where $f(\boldsymbol{Y}) \equiv \int f(\boldsymbol{Y} \mid \theta) f(\theta)\, d\theta$.



Source: Wikipedia

Bayes' rule (cont.)

$$f(\theta \mid \boldsymbol{Y}) = \frac{f(\boldsymbol{Y} \mid \theta) f(\theta)}{f(\boldsymbol{Y})}$$

- This is really just the definition of conditional densities.
  - The denominator $f(\boldsymbol{Y}) \equiv \int f(\boldsymbol{Y} \mid \theta) f(\theta)\, d\theta$ makes the posterior density integrate to 1.
- The formula also makes intuitive sense: relative to the prior, we up-weight those values θ that have a high "probability" of generating the observed data $\boldsymbol{Y}$, i.e., high likelihood value $f(\boldsymbol{Y} \mid \theta)$.
- The posterior synthesizes two pieces of information: prior knowledge and the new data.
  - Neither piece is infallible: the data has noise (sampling error) and our prior could be wrong. So we should not rely exclusively on either piece.

AR(1) example

- Let's do an analytical example to see the mechanics of Bayesian inference (though this course will emphasize computation).
- Consider an AR(1) model with normally distributed shocks:

$$y_t = \rho y_{t-1} + u_t, \quad u_t \overset{iid}{\sim} N(0, \sigma^2).$$

  For simplicity, we omit the intercept and assume σ is *known*.

- Prior distribution for the unknown ρ:

$$\rho \sim N(m, \lambda^2).$$

  - Centered at $\rho = m$. This is our "best guess" for ρ *a priori*.
  - Larger $\lambda \Longrightarrow$ more prior uncertainty.

- Prior density:

$$f(\rho) = \frac{1}{\sqrt{2\pi\lambda^2}} e^{-(\rho-m)^2/(2\lambda^2)}.$$

AR(1) example: likelihood function

- Next, let's derive the likelihood function for a data set $\boldsymbol{Y} \equiv (y_1, y_2, \ldots, y_T)'$ of size T.
- By definition of conditional densities,

$$f(y_1, y_2, y_3 \mid \rho) = f(y_3 \mid y_2, y_1, \rho) f(y_2 \mid y_1, \rho) f(y_1 \mid \rho).$$

- Generalizing:

$$f(\boldsymbol{Y} \mid \rho) = f(y_1, \ldots, y_T \mid \rho) = f(y_1 \mid \rho) \prod_{t=2}^{T} f(y_t \mid y_{t-1}, y_{t-2}, \ldots, y_1, \rho).$$

AR(1) example: likelihood function (cont.)

- In the Gaussian AR(1) model,

$$(y_t \mid y_{t-1}, y_{t-2}, \ldots, y_1, \rho) \sim N(\rho y_{t-1}, \sigma^2),$$

corresponding to the conditional density

$$f(y_t \mid y_{t-1}, \rho) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y_t - \rho y_{t-1})^2/(2\sigma^2)}.$$

- Then the likelihood equals

$$\begin{aligned} f(\boldsymbol{Y} \mid \rho) &= f(y_1 \mid \rho) \prod_{t=2}^{T} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(y_t - \rho y_{t-1})^2/(2\sigma^2)} \\ &= f(y_1 \mid \rho)(2\pi\sigma^2)^{-(T-1)/2} e^{\sum_{t=2}^{T} -(y_t - \rho y_{t-1})^2/(2\sigma^2)}. \end{aligned}$$

- It's conventional to ignore the first factor $f(y_1 \mid \rho)$ for tractability.

AR(1) example: posterior

- Bayes' rule:

$$
\begin{aligned}
f(\rho \mid \boldsymbol{Y}) &= \frac{\frac{1}{\sqrt{2\pi\lambda^2}} e^{-(\rho-m)^2/(2\lambda^2)} \left(2\pi\sigma^2\right)^{-(T-1)/2} e^{\sum_{t=2}^{T} -(y_t-\rho y_{t-1})^2/(2\sigma^2)}}{f(\boldsymbol{Y})} \\
&= C_1 \times e^{-(\rho-m)^2/(2\lambda^2) - \sum_{t=2}^{T}(y_t-\rho y_{t-1})^2/(2\sigma^2)} \\
&= C_2 \times e^{-(\rho-\bar{\rho})^2/(2\bar{\tau}^2)},
\end{aligned}
$$

where C_1, C_2 are constants that don't depend on ρ,

$$
\bar{\rho} \equiv \frac{\bar{\tau}^2}{\lambda^2} m + \left(1 - \frac{\bar{\tau}^2}{\lambda^2}\right) \hat{\rho}, \quad \bar{\tau}^2 \equiv \frac{1}{\lambda^{-2} + \sum_{t=2}^{T} y_{t-1}^2/\sigma^2},
$$

and $\hat{\rho} \equiv \sum_{t=2}^{T} y_t y_{t-1} / \sum_{t=2}^{T} y_{t-1}^2$ is the usual OLS estimator.

- Exercise (tedious): verify the last step.

AR(1) example: posterior (cont.)

$$f(\rho \mid \boldsymbol{Y}) = C_2 \times e^{-(\rho-\bar{\rho})^2/(2\bar{\tau}^2)}$$

- We recognize this as the density of a normal distribution with mean $\bar{\rho}$ and variance $\bar{\tau}^2$.
  - For the density to integrate to 1, we must have $C_2 = (2\pi\bar{\tau}^2)^{-1/2}$.
- So the posterior distribution for ρ is normal:

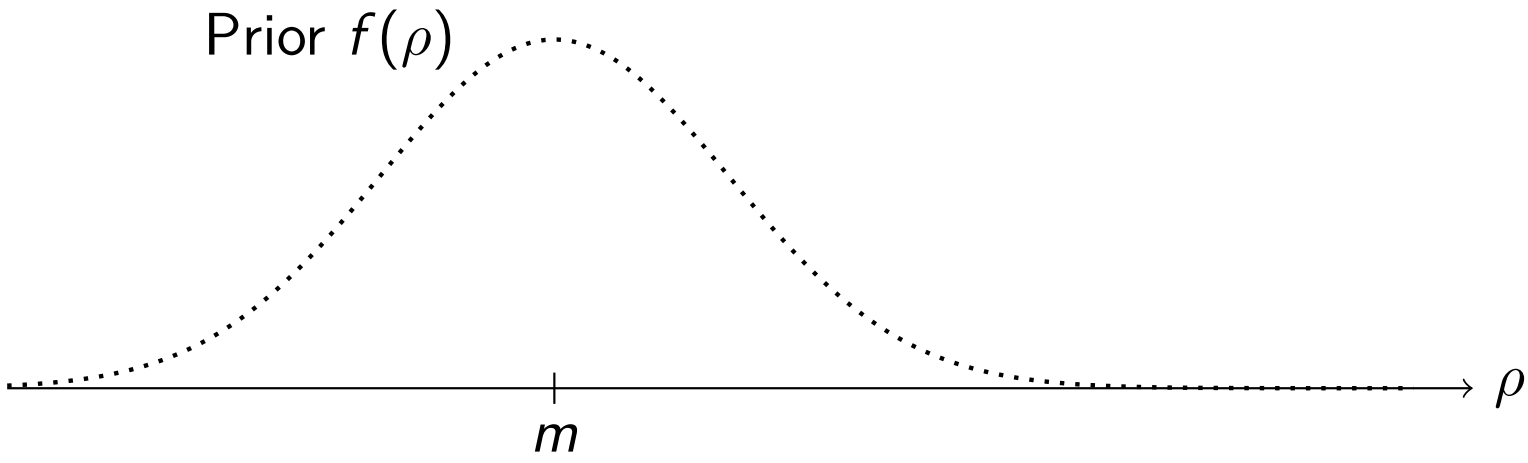
$$\boxed{(\rho \mid \boldsymbol{Y}) \sim N\left(\bar{\rho}, \bar{\tau}^2\right)}.$$

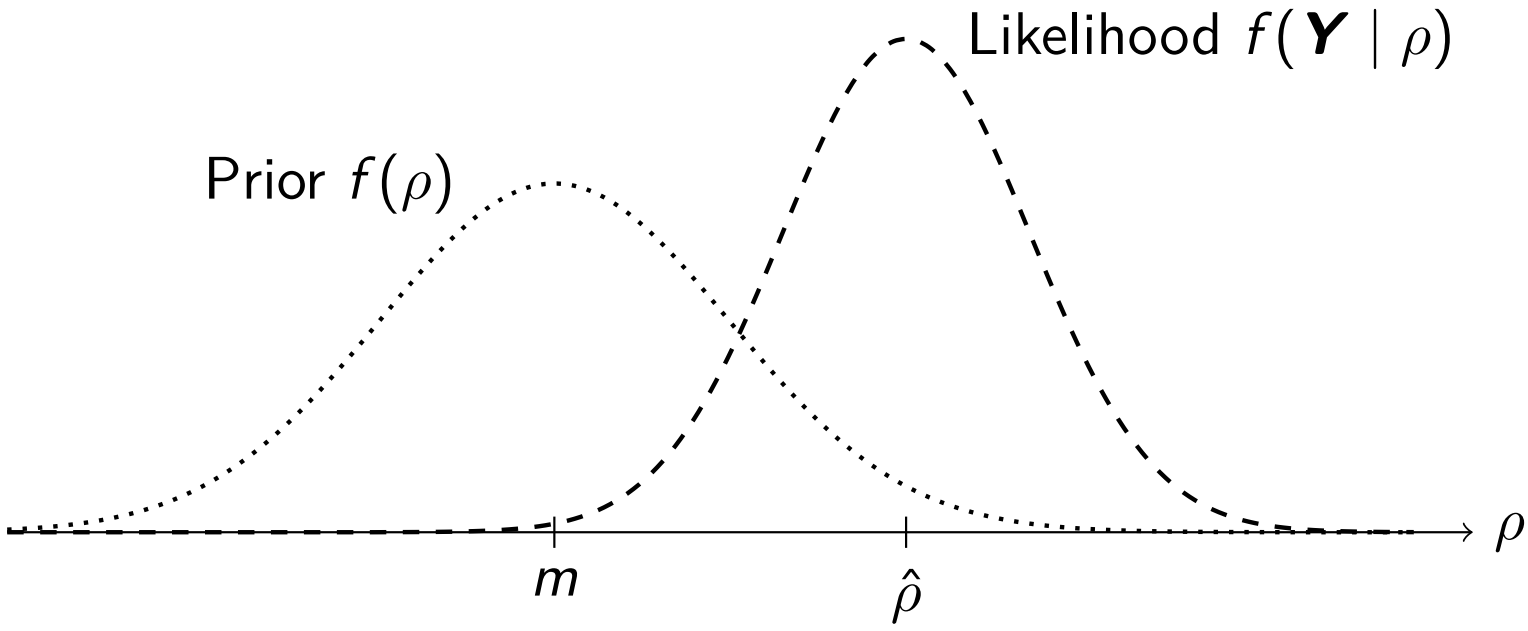
- The data has caused us to update the prior mean m to the posterior mean $E[\rho \mid \boldsymbol{Y}] = \bar{\rho}$, and the prior variance λ^2 to the posterior variance $\text{Var}(\rho \mid \boldsymbol{Y}) = \bar{\tau}^2$.

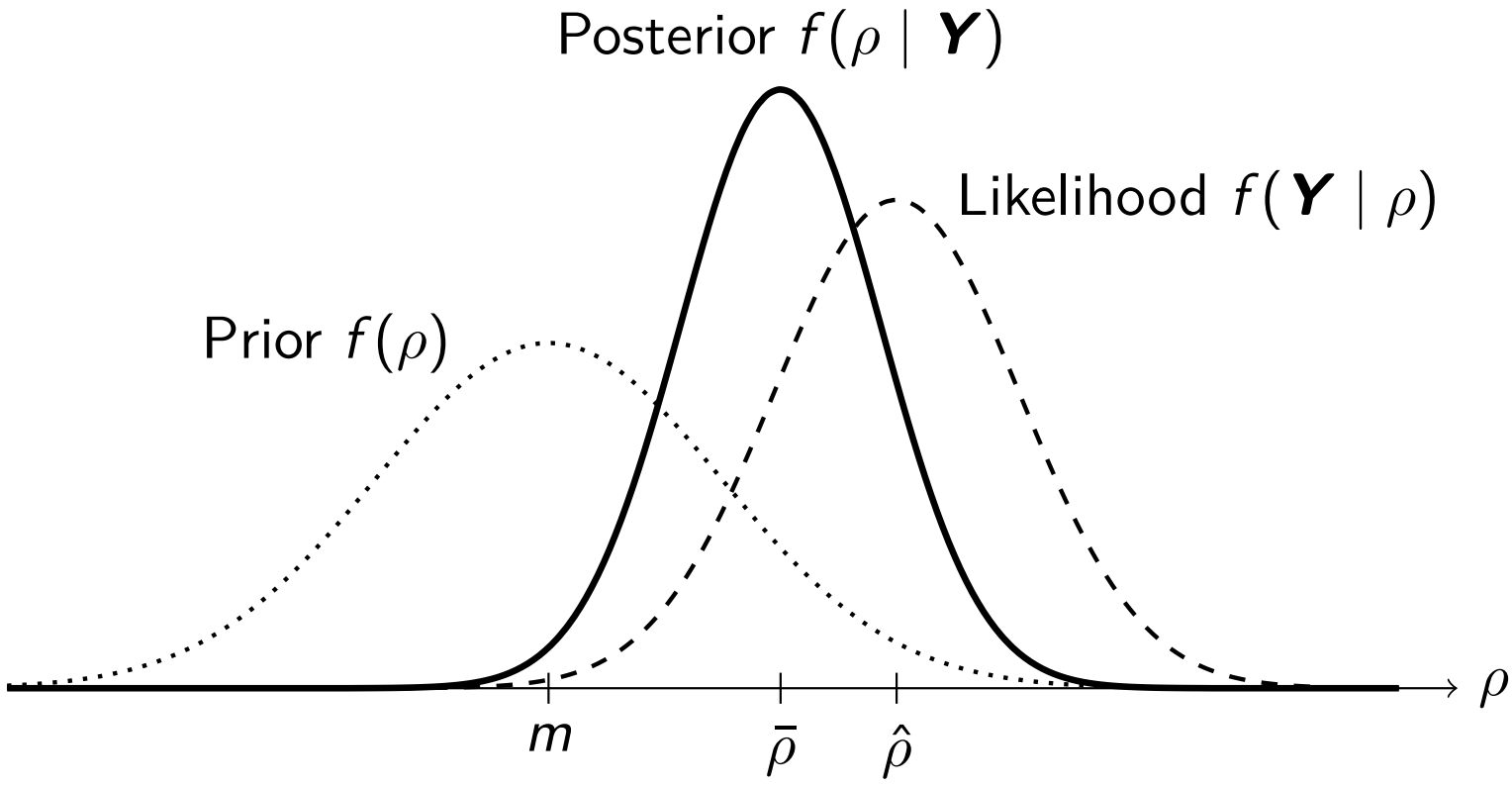
AR(1) example: posterior mean

$$\bar{\rho} \equiv \frac{\bar{\tau}^2}{\lambda^2} m + \left(1 - \frac{\bar{\tau}^2}{\lambda^2}\right) \hat{\rho}, \quad \bar{\tau}^2 \equiv \frac{1}{\lambda^{-2} + \sum_{t=2}^{T} y_{t-1}^2 / \sigma^2}$$

- The posterior mean $E[\rho \mid \boldsymbol{Y}] = \bar{\rho}$ is a weighted average of the prior mean m and the OLS estimator $\hat{\rho}$.
  - The Bayesian estimator $\bar{\rho}$ **shrinks** the frequentist estimator $\hat{\rho}$ toward the prior mean m.
- Weight $\bar{\tau}^2/\lambda^2 \in [0, 1]$ on prior mean $=$ ratio of the posterior variance to the prior variance of ρ.
- Exercise: show that the weight on the prior mean converges to 0 (so $\bar{\rho} \to \hat{\rho}$) if either...
  - $\lambda^2 \to \infty$ (the data dominates when our prior is very uncertain/**diffuse**).
  - $T \to \infty$ (the data dominates the prior in large samples).


Prior $f(\rho)$
m
ρ


Likelihood $f(\boldsymbol{Y} \mid \rho)$
Prior $f(\rho)$
ρ
m
$\hat{\rho}$


Posterior $f(\rho \mid \boldsymbol{Y})$
Likelihood $f(\boldsymbol{Y} \mid \rho)$
Prior $f(\rho)$
ρ
m
$\bar{\rho}$
$\hat{\rho}$

AR(1) example: credible interval

- Let Q_α be the α-quantile of the standard normal distribution.
- Recall the posterior distribution:

$$(\rho \mid \boldsymbol{Y}) \sim N\left(\bar{\rho}, \bar{\tau}^2\right).$$

- Then

$$\boxed{P\left(\rho \in \left[\bar{\rho} - Q_{1-\alpha/2}\bar{\tau}, \bar{\rho} + Q_{1-\alpha/2}\bar{\tau}\right] \;\middle|\; \boldsymbol{Y}\right) = 1 - \alpha}.$$

  This interval is called an (equal-tailed) $1 - \alpha$ **credible interval** for ρ.

- A Bayesian can say: the probability that ρ lies in this *particular* interval is $1 - \alpha$, given the data that we *actually observed*.
  - Contrasts with the interpretation of a frequentist confidence interval!

Summarizing the posterior in general models

$$f(\theta \mid \boldsymbol{Y}) = \frac{f(\boldsymbol{Y} \mid \theta) f(\theta)}{f(\boldsymbol{Y})}$$

- For some models and priors, the posterior density does not have a simple, known form (e.g., normal).
- It's typically easy to evaluate the prior density $f(\theta)$ and the likelihood $f(\boldsymbol{Y} \mid \theta)$. But the **marginal density** $f(\boldsymbol{Y}) \equiv \int f(\boldsymbol{Y} \mid \theta) f(\theta)\, d\theta$ may be intractable.
- Similarly, it's often intractable to directly evaluate posterior summaries such as the mean

$$E[\theta \mid \boldsymbol{Y}] = \int \theta f(\theta \mid \boldsymbol{Y})\, d\theta.$$

- While standard numerical integration could work if θ has dimension 1 or 2, it won't work in realistic models with many parameters.

Summarizing the posterior in general models (cont.)

$$f(\theta \mid \boldsymbol{Y}) = \frac{f(\boldsymbol{Y} \mid \theta) f(\theta)}{f(\boldsymbol{Y})}$$

- However, suppose we found a way to randomly **sample** from the posterior distribution without having to derive it analytically or directly evaluate high-dimensional integrals.
- If we manage to generate draws $\theta_{(0)}, \theta_{(1)}, \ldots, \theta_{(B)}$ from the posterior distribution on our computer, we can summarize the distribution by computing their sample mean, median, percentiles, etc.
- For example,

$$E[\theta \mid \boldsymbol{Y}] \approx \frac{1}{B+1} \sum_{b=0}^{B} \theta_{(b)}.$$

MCMC

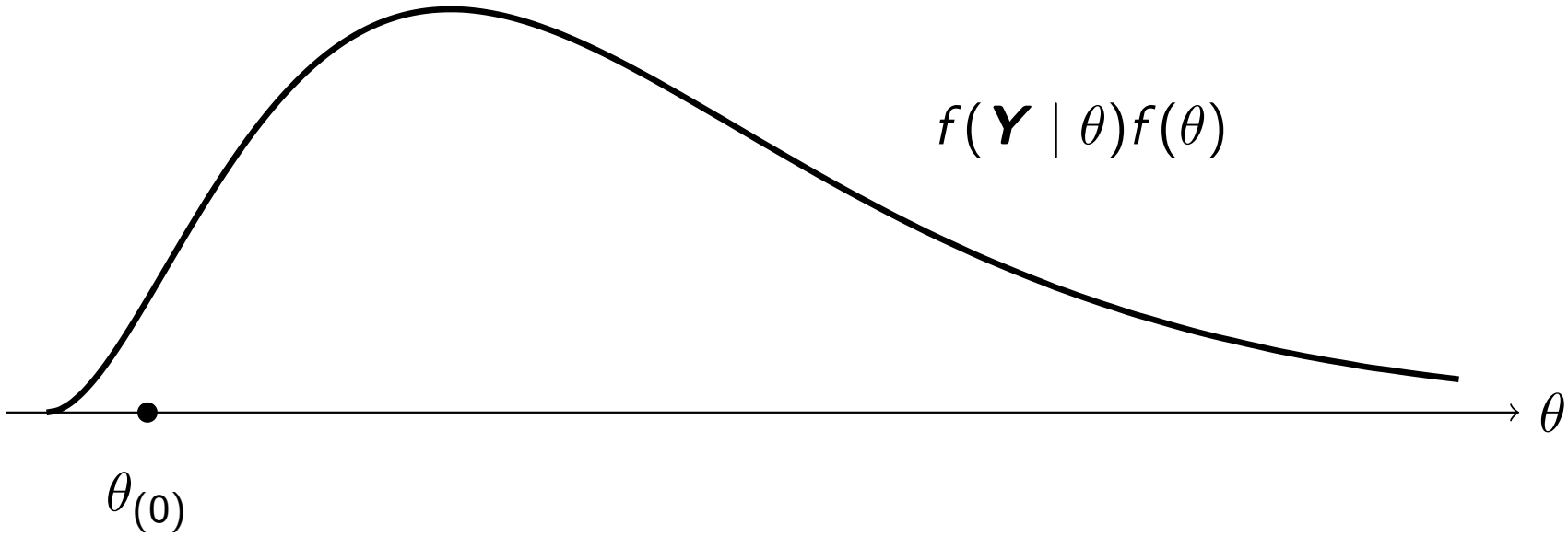
$$f(\theta \mid \boldsymbol{Y}) = \frac{f(\boldsymbol{Y} \mid \theta) f(\theta)}{f(\boldsymbol{Y})}$$

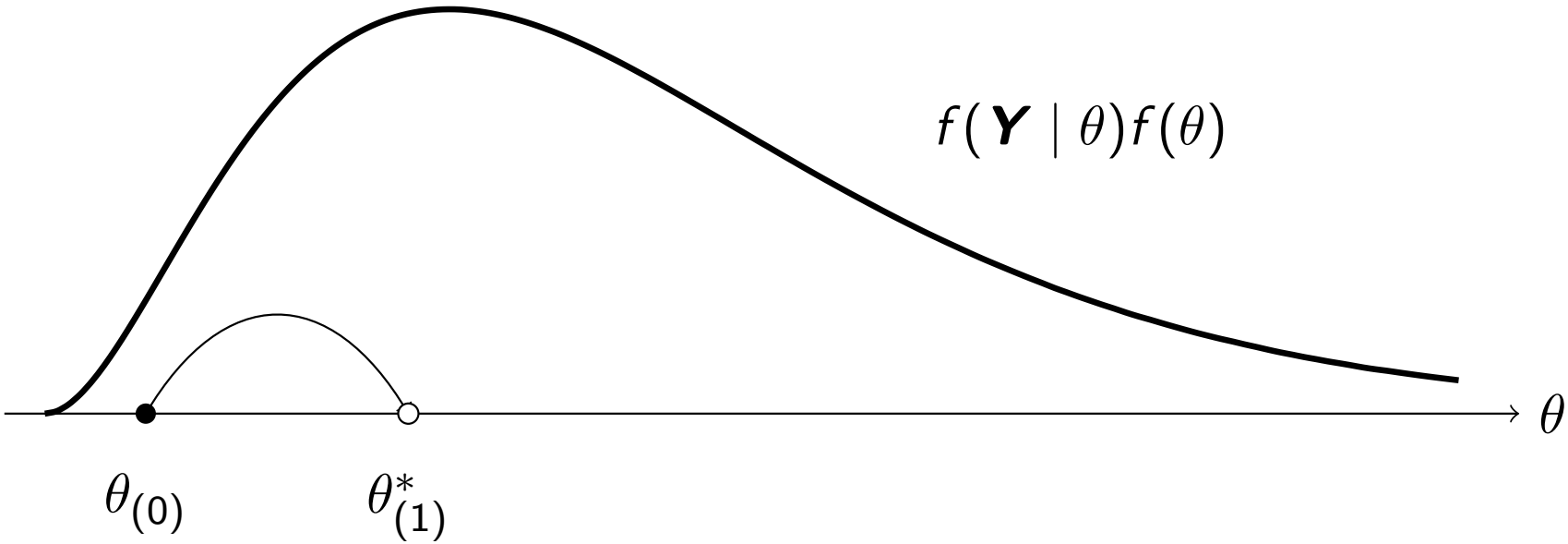
- **Markov Chain Monte Carlo** (MCMC) methods are designed to randomly sample from the distribution whose density is proportional to the numerator $f(\boldsymbol{Y} \mid \theta) f(\theta)$.
- They do this by stochastically "walking around" in the parameter space for θ, tending to prefer steps that increase $f(\boldsymbol{Y} \mid \theta) f(\theta)$.
- But the algorithm also sometimes moves in directions that *decrease* $f(\boldsymbol{Y} \mid \theta) f(\theta)$, so as not to get stuck at a single point (which would not explore the full distribution).

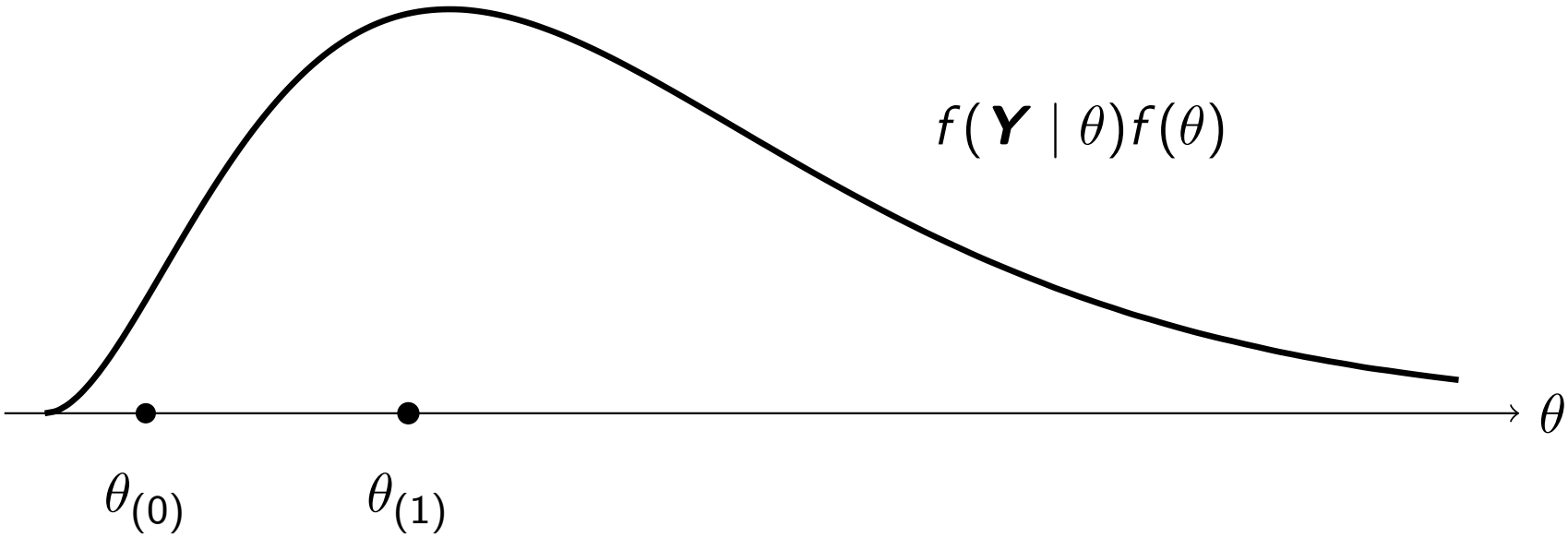
MCMC (cont.)

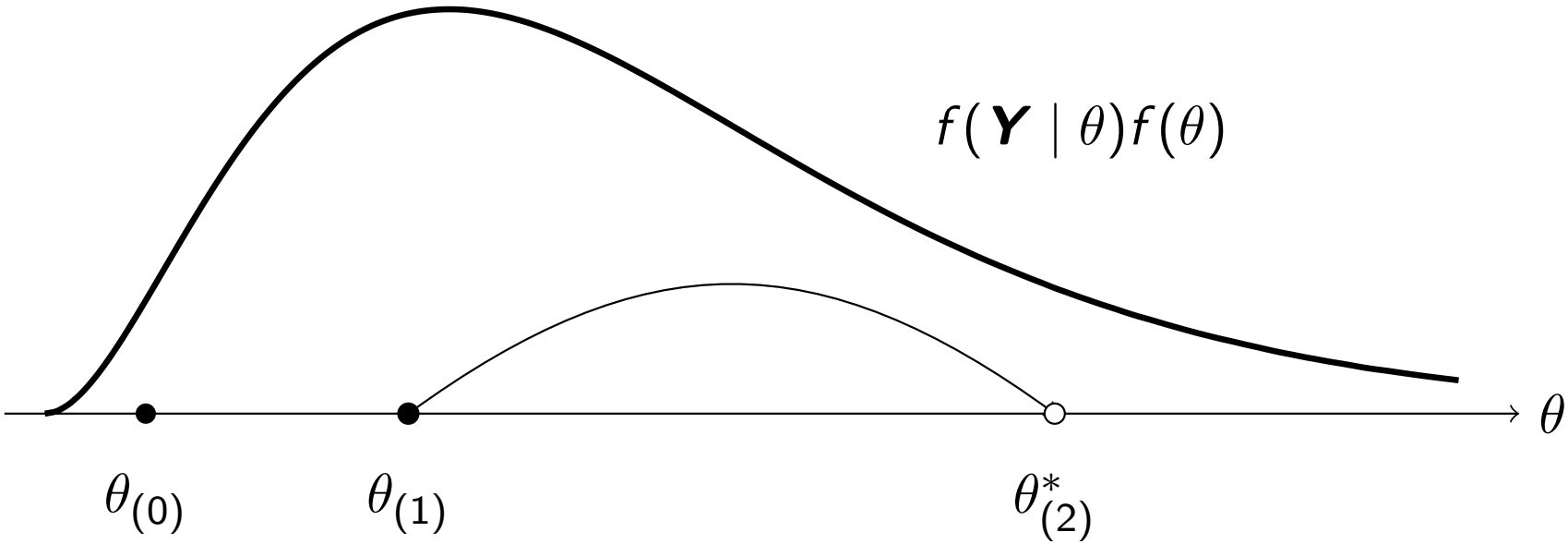
- Prototypical MCMC algorithm:
  - Draw an initial parameter vector $\theta_{(0)}$.
  - For each iteration $b = 1, \ldots, B$:
    1. **Propose** a new draw $\theta^*_{(b)}$.
    2. **Accept** this draw and set $\theta_{(b)} = \theta^*_{(b)}$ with a probability that's increasing in $f(\boldsymbol{Y} \mid \theta) f(\theta)$. Otherwise, **reject** the draw and stay put: $\theta_{(b)} = \theta_{(b-1)}$.
- This generates a serially correlated time series of draws $\theta_{(0)}, \theta_{(1)}, \ldots, \theta_{(B)}$. Called an MCMC **chain**.
- The proposals and acceptance probability are constructed in an ingenious way to ensure that the chain has a stationary distribution whose density is exactly our target posterior density

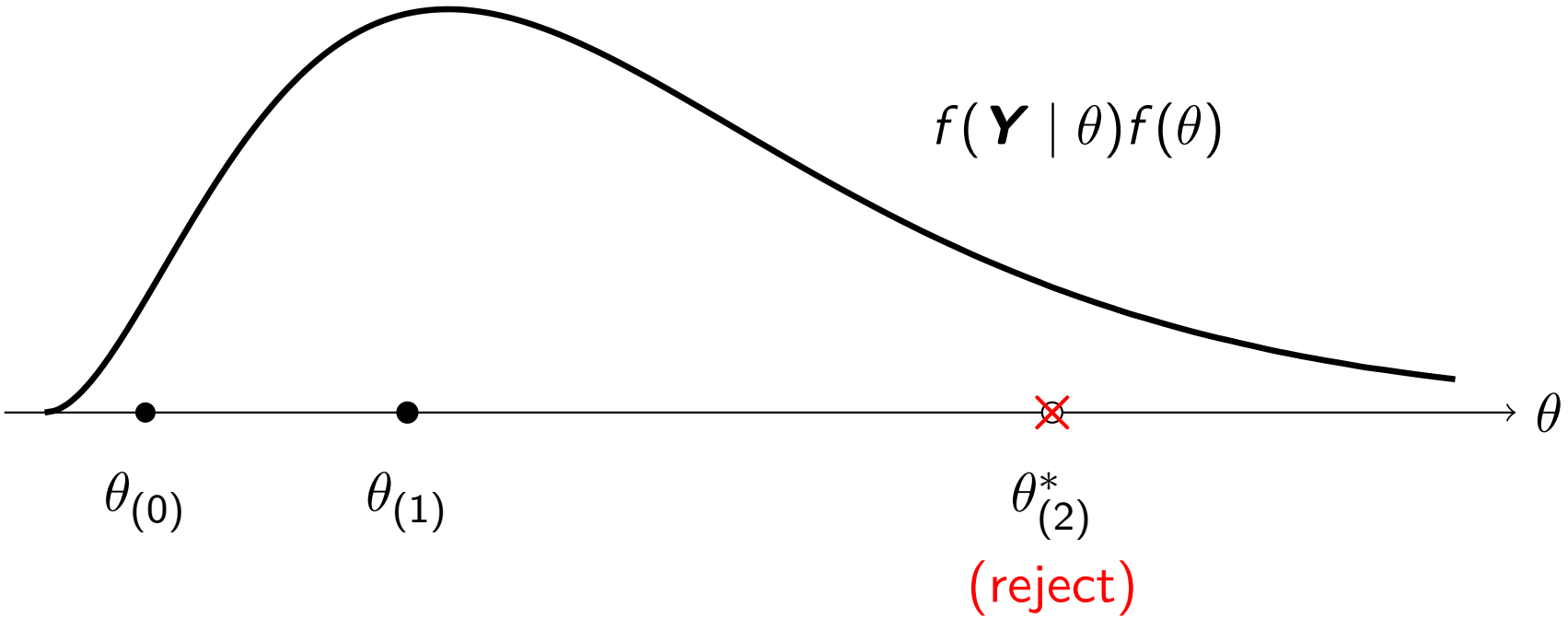
$$f(\theta \mid \boldsymbol{Y}) = \frac{f(\boldsymbol{Y} \mid \theta) f(\theta)}{f(\boldsymbol{Y})}.$$

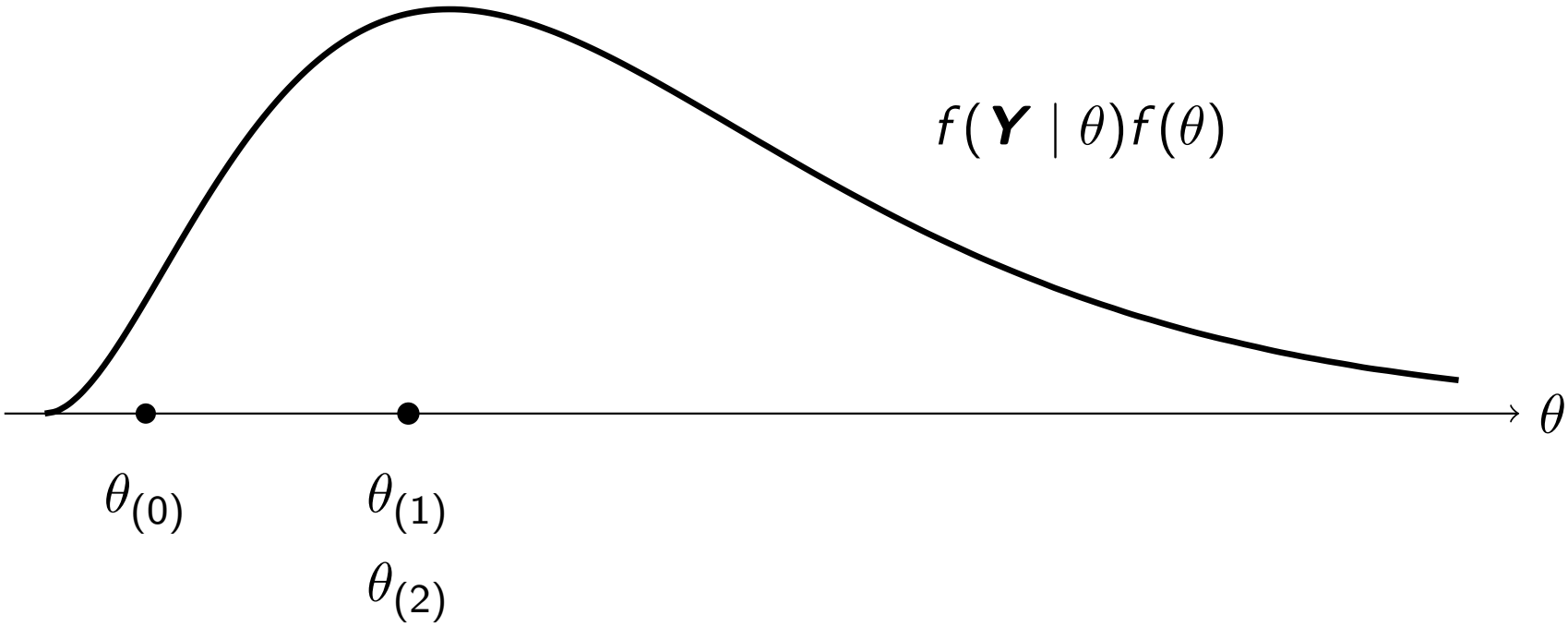

$f(\boldsymbol{Y} \mid \theta) f(\theta)$
θ
$\theta_{(0)}$

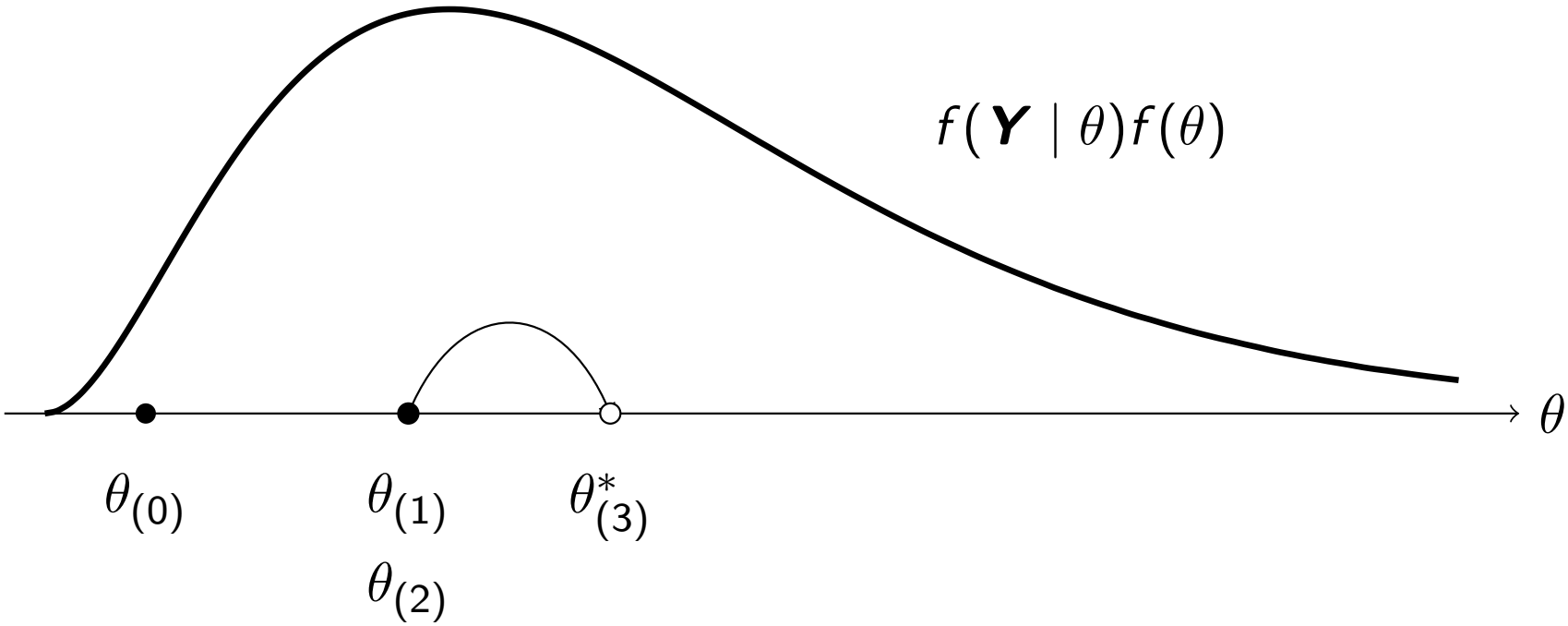

$f(\boldsymbol{Y} \mid \theta)f(\theta)$
θ
$\theta_{(0)}$
$\theta^*_{(1)}$

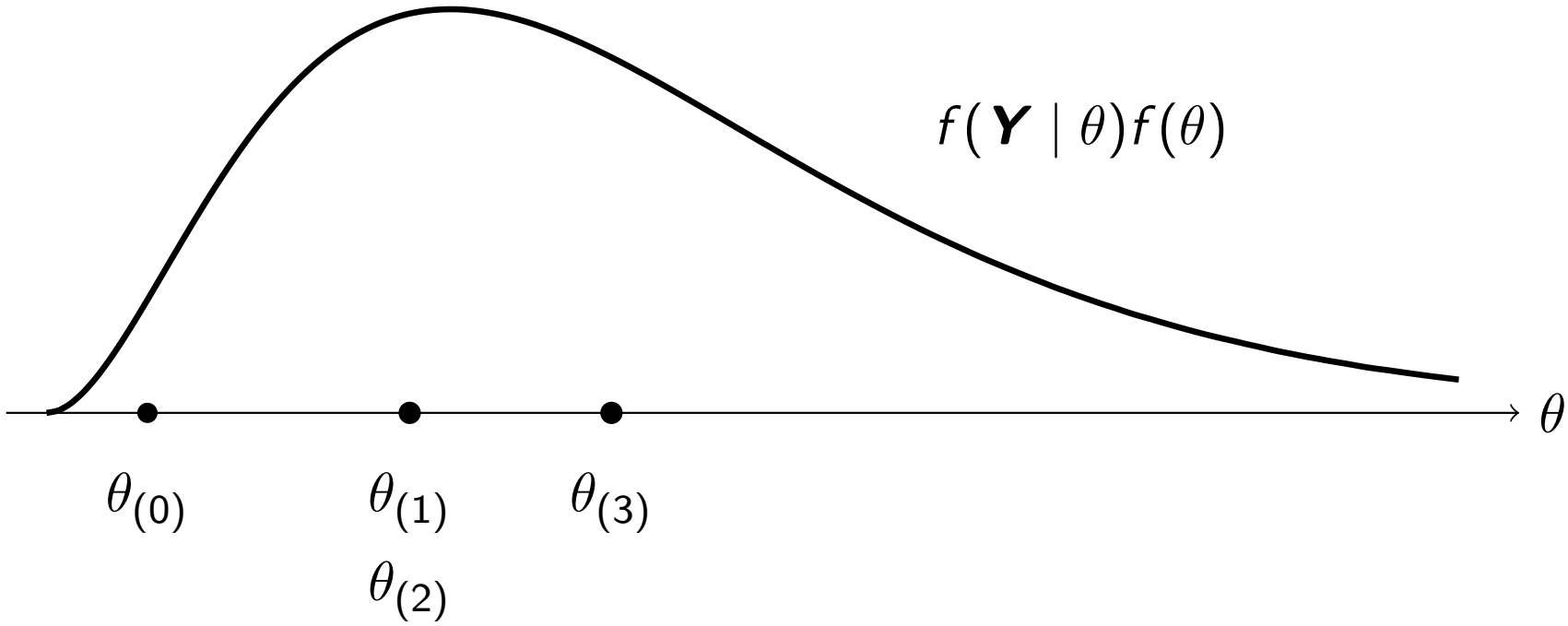

$f(\boldsymbol{Y} \mid \theta)f(\theta)$
θ
$\theta_{(0)}$
$\theta_{(1)}$

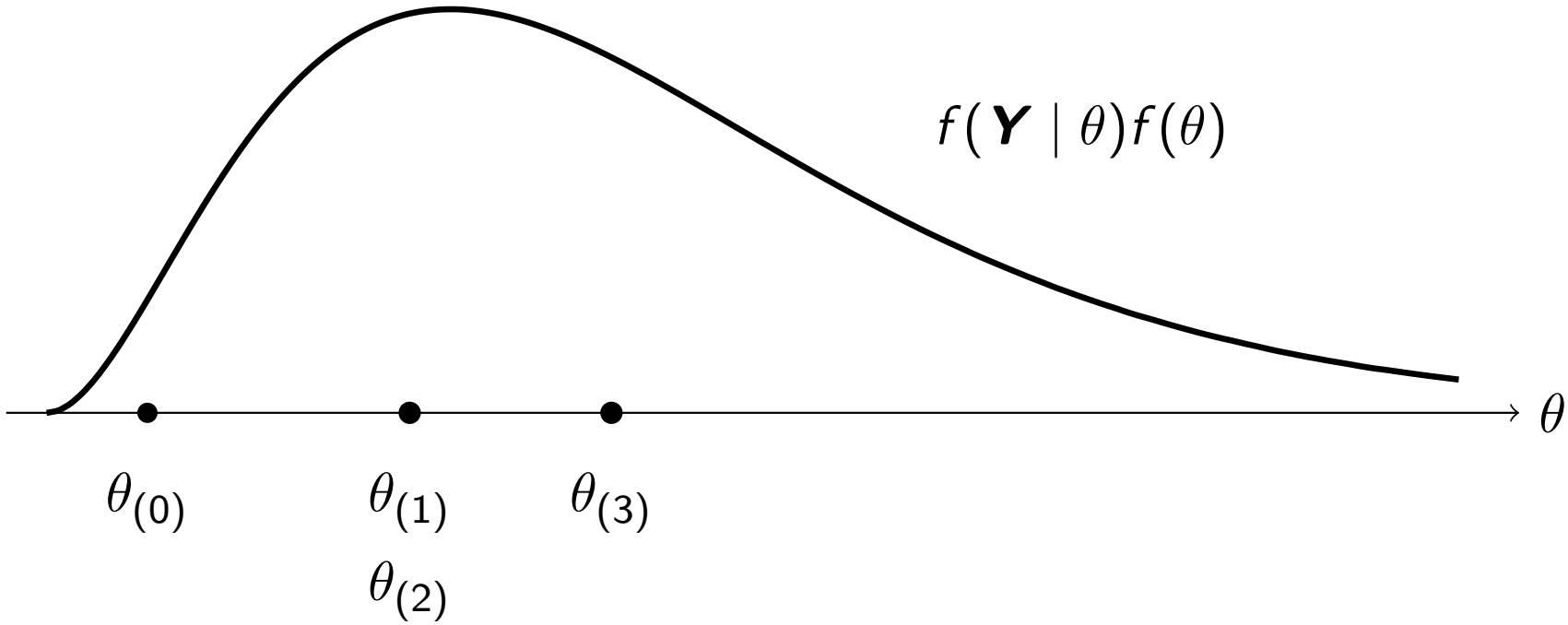

$f(\boldsymbol{Y} \mid \theta)f(\theta)$
θ
$\theta_{(0)}$
$\theta_{(1)}$
$\theta^*_{(2)}$


$f(\boldsymbol{Y} \mid \theta)f(\theta)$
θ
$\theta_{(0)}$
$\theta_{(1)}$
$\theta^*_{(2)}$
(reject)


$f(\boldsymbol{Y} \mid \theta)f(\theta)$
θ
$\theta_{(0)}$
$\theta_{(1)}$
$\theta_{(2)}$


$f(\boldsymbol{Y} \mid \theta)f(\theta)$
θ
$\theta_{(0)}$
$\theta_{(1)}$
$\theta^*_{(3)}$
$\theta_{(2)}$


$f(\boldsymbol{Y} \mid \theta)f(\theta)$
θ
$\theta_{(0)}$
$\theta_{(1)}$
$\theta_{(3)}$
$\theta_{(2)}$



- Since the algorithm is more likely to accept proposals that increase the (un-normalized) density $f(\boldsymbol{Y} \mid \theta)f(\theta)$, most draws will be in the region with high density.
- But we do sometimes accept proposals with lower density, so we also get draws in the low-density region.

MCMC (cont.)

- The distribution of the draws in the chain eventually **converges** to the stationary distribution (i.e., the posterior distribution) as $B \to \infty$.
- However, if the initial parameter $\theta_{(0)}$ is far from the center of the stationary distribution, it may take many iterations to converge.
  - This is just like how a stationary AR(1) process depends on the initial value y_0 at first, but as time goes on, the effect of the initial value vanishes.
- So in practice we discard the first several steps from the MCMC chain. The discarded draws are called **burn-in**.
- After burn-in, the chain ought to look stationary.

Stan

- Designing efficient MCMC procedures is hard and very technical work.
- The Bayesian software **Stan** automates this process.
  - Uses a variant of MCMC called "Hamiltonian Monte Carlo" (HMC).
  - Can be called from several programming languages, such as Python, R, Matlab, and Julia.
  - The documentation is well-written, with many examples.
- Researchers just need to supply a "model file" specifying the prior and likelihood function in natural mathematical notation.
- Then Stan figures out how to do MCMC sampling on its own.
- After sampling, Stan generates posterior summaries and diagnostics.

Bayesian estimation of an AR(p) model

- We'll apply Stan to an AR(p) model with normally distributed shocks:

$$y_t = c + a_1 y_{t-1} + \cdots + a_p y_{t-p} + u_t, \quad u_t \overset{iid}{\sim} N(0, \sigma^2).$$

- We would like to estimate this with Bayesian methods.
  - This particular model is actually not hard to analyze analytically, but Stan can be useful also in more difficult models, as we'll see.
- Data set: $\boldsymbol{Y} \equiv (y_1, y_2, \ldots, y_T)'$.
- Parameter vector: $\boldsymbol{\theta} \equiv (c, a_1, \ldots, a_p, \sigma)'$. Dimension $(p+2) \times 1$.

Bayesian AR(p): likelihood

- In this model,

$$(y_t \mid y_{t-1}, y_{t-2}, \ldots, y_1, \boldsymbol{\theta}) \sim N(c + \boldsymbol{x}_t'\boldsymbol{\beta}, \sigma^2),$$

$$\text{where} \quad \boldsymbol{x}_t \equiv \underbrace{(y_{t-1}, \ldots, y_{t-p})'}_{p\times 1}, \quad \boldsymbol{\beta} \equiv \underbrace{(a_1, a_2, \ldots, a_p)'}_{p\times 1}.$$

- Similar to the AR(1) case,

$$f(\boldsymbol{Y} \mid \boldsymbol{\theta}) = f(y_1, \ldots, y_p \mid \boldsymbol{\theta}) \prod_{t=p+1}^{T} f(y_t \mid y_{t-1}, y_{t-2}, \ldots, y_1, \boldsymbol{\theta})$$

$$= f(y_1, \ldots, y_p \mid \boldsymbol{\theta})(2\pi\sigma^2)^{-(T-p)/2} e^{-\sum_{t=p+1}^{T}(y_t - c - \boldsymbol{x}_t'\boldsymbol{\beta})^2/(2\sigma^2)}.$$

- It's conventional to ignore the first factor $f(y_1, \ldots, y_p \mid \boldsymbol{\theta})$.

Bayesian AR(p): likelihood (cont.)

$$f(\boldsymbol{Y} \mid \boldsymbol{\theta}) = (2\pi\sigma^2)^{-(T-p)/2} e^{-\sum_{t=p+1}^{T}(y_t - c - \boldsymbol{x}_t'\beta)^2/(2\sigma^2)}$$

- This likelihood function corresponds to the density of the $(T-p)\times 1$ normal random vector $\tilde{\boldsymbol{Y}} \equiv (y_{p+1}, \ldots, y_T)'$:

$$\tilde{\boldsymbol{Y}} \sim N\left(c\mathbf{1} + \boldsymbol{X}\beta, \sigma^2 \boldsymbol{I}\right),$$

  where $\boldsymbol{I}$ is the $(T-p)\times(T-p)$ identity matrix, $\mathbf{1}$ is a $(T-p)\times 1$ vector of 1's, and

$$\boldsymbol{X} \equiv \underbrace{\left(\boldsymbol{x}_{p+1}, \boldsymbol{x}_{p+2}, \ldots, \boldsymbol{x}_T\right)'}_{(T-p)\times p}.$$

- We exploit this in the Stan model file below.

Bayesian AR(p): prior

- Often when forecasting macro time series, it's surprisingly hard to beat the naive no-change forecast: $y_{t+1|t} = y_t$.
- In the AR(p) model, the no-change forecast is optimal for the parameters $a_1 = 1$ and $a_\ell = 0$ for $\ell \geq 2$, yielding the random walk

$$y_t = y_{t-1} + u_t.$$

- This suggests a prior distribution where

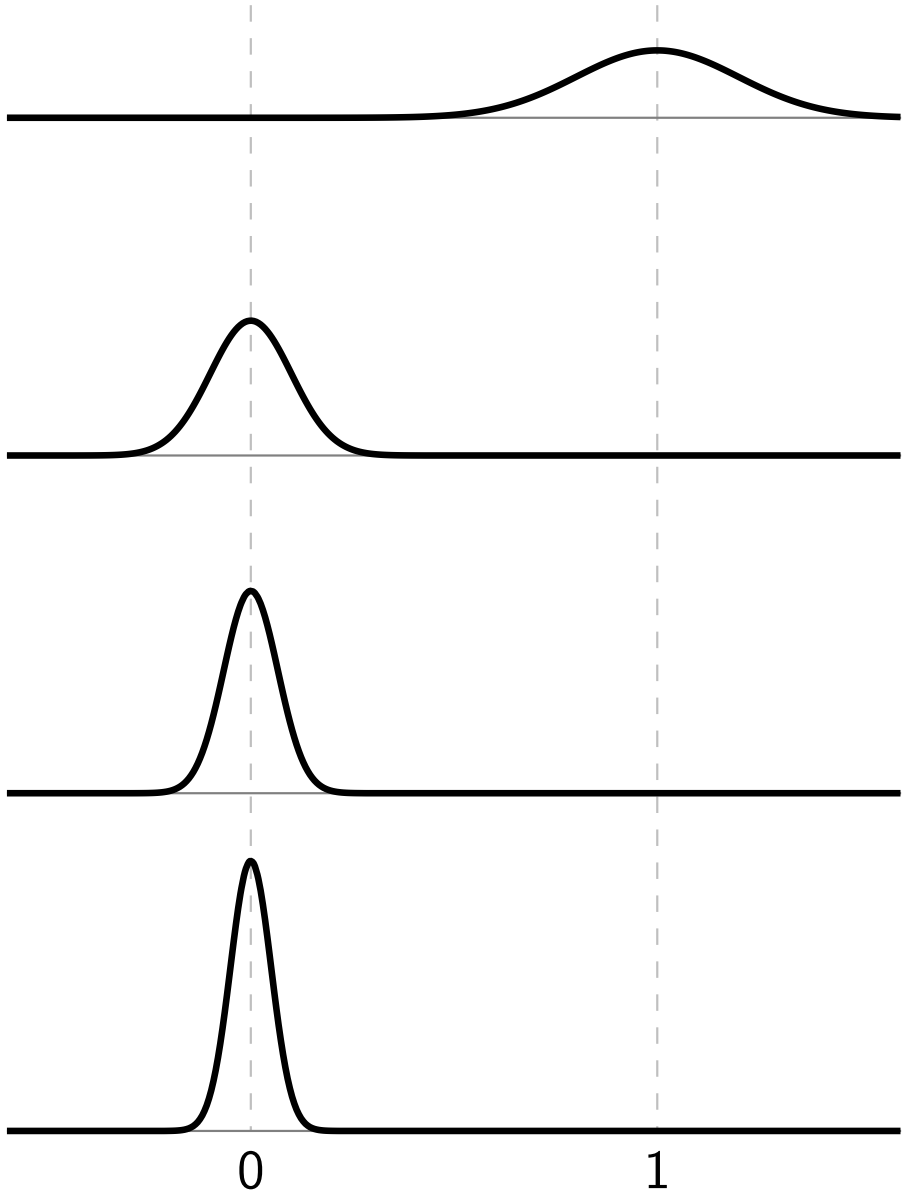
$$E[a_1] = 1, \quad E[a_2] = E[a_3] = \cdots = E[a_p] = 0.$$

Bayesian AR(p): prior (cont.)

- The **Minnesota prior** (invented by researchers at University of Minnesota) specifies

$$a_\ell \sim N\left(m_\ell, \frac{\lambda^2}{\ell^2}\right), \quad \text{where} \quad m_1 \equiv 1, \quad m_2 = \cdots = m_p = 0.$$

- "Shrinks" the coefficient a_1 towards 1 and $a_2, \ldots, a_p$ towards 0.
- More shrinkage (due to smaller prior variance) at longer lags ℓ.
  - Effectively reduces the dimension of the parameter space. So we can use a large lag order p without overfitting.
  - For large ℓ, $|\hat{a}_\ell|$ must be very large to shift the posterior away from 0.
- λ^2 governs the overall degree of shrinkage.
  - Common choice in literature: $\lambda = 0.2$.



$a_1 \sim N(1, \lambda^2)$

$a_2 \sim N(0, (\lambda/2)^2)$

$a_3 \sim N(0, (\lambda/3)^2)$

$a_4 \sim N(0, (\lambda/4)^2)$

Bayesian AR(p): prior (cont.)

- For the intercept and the shock standard deviation it's common to use very diffuse priors, such as

$$c \sim N\left(0, 10^4 \hat{\sigma}_y^2\right),$$

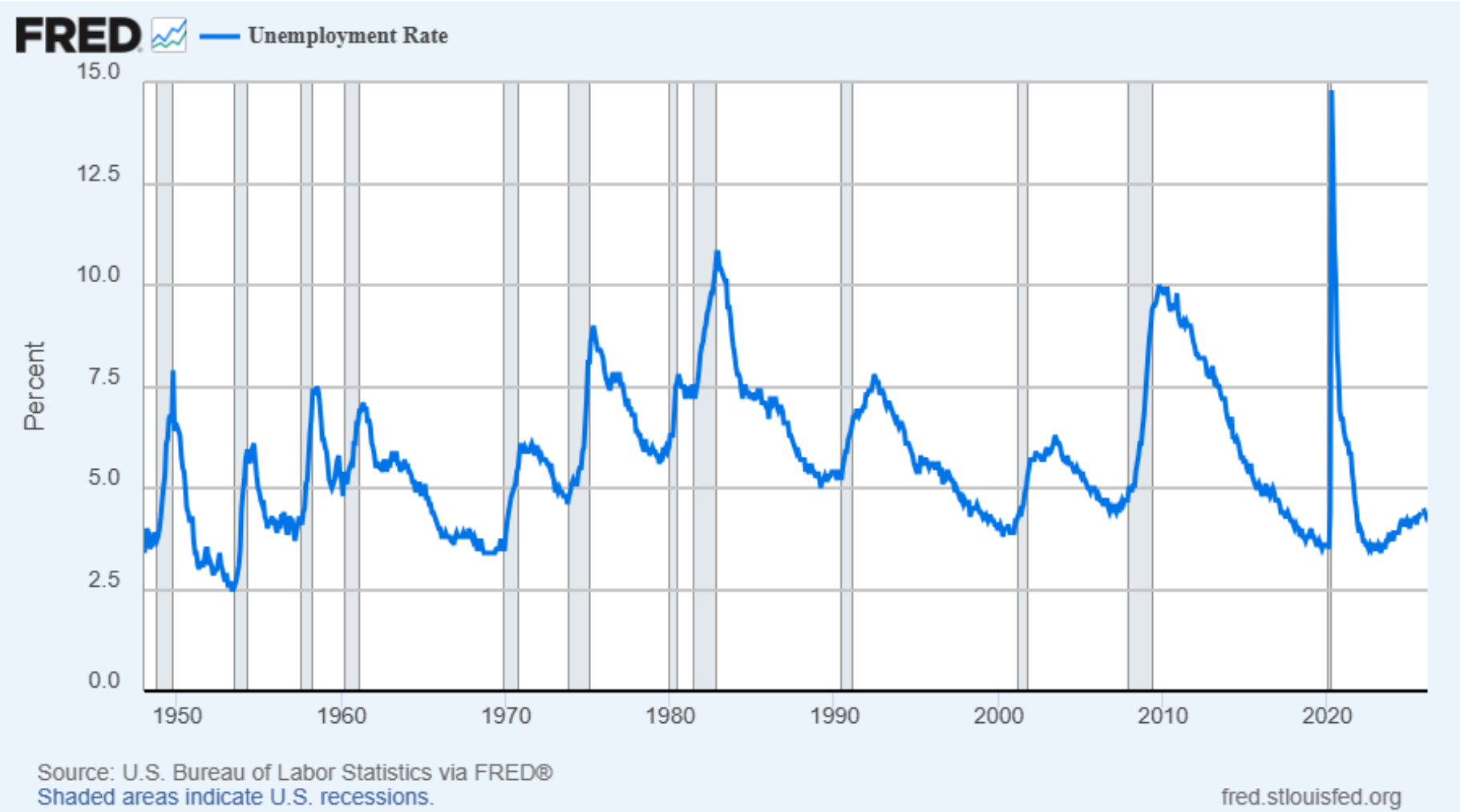
$$\sigma \sim \text{Cauchy}_+(0, \hat{\sigma}_y),$$

  where $\hat{\sigma}_y^2$ is the sample variance of $\{y_t\}$.

  - "Cauchy$_+$" is a Cauchy distribution restricted to the positive real line (a very fat-tailed distribution).

- Finally, we assume all $p + 2$ parameters in $\boldsymbol{\theta}$ are independent *a priori*: their joint prior density is the product of the marginal densities.

- We now have the likelihood and the prior. Let's crank up Stan!



Discard data from 2020 onwards (we'll discuss how to model the massive Covid shock in the next slide deck)

My Stan model file `slides12_ar.stan`:

```
data {
  int<lower=1> T;              // effective sample size
  int<lower=1> p;              // lag order
  vector[T] y;                 // observed series
  matrix[T, p] X;              // lag matrix
  real<lower=0> lambda;        // prior tightness
  real<lower=0> scale_sigma;   // prior scale for sigma
  real<lower=0> scale_c;       // prior scale for intercept c
}

parameters {
  real c;                      // intercept
  vector[p] a;                 // AR coefficients
  real<lower=0> sigma;         // innovation std. dev.
}
```

(continued next slide)

```
model {
  // Minnesota prior on AR coefficients
  for (l in 1:p) {
    real m_ = (l == 1) ? 1.0 : 0.0;
    real sd_ = lambda / l;
    a[l] ~ normal(m_, sd_);
  }

  // Intercept prior
  c ~ normal(0, scale_c);

  // Innovation std. dev.
  sigma ~ cauchy(0, scale_sigma);

  // Likelihood (vectorized)
  y ~ normal(c + X * a, sigma);
}
```

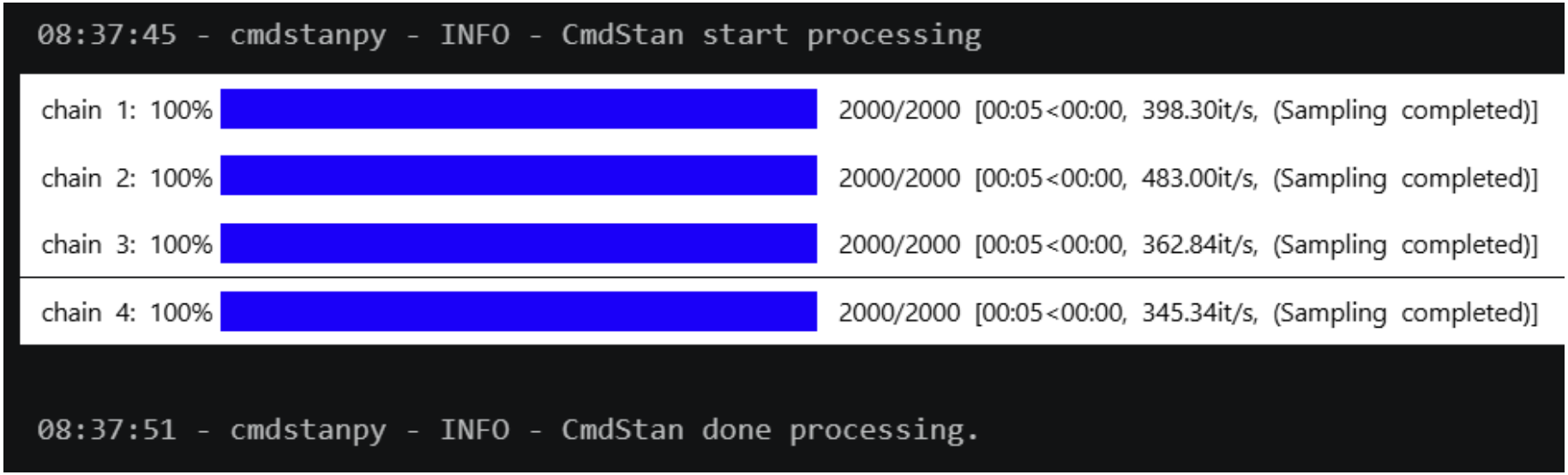
Note: Stan (like `scipy`) parametrizes $N(0, \sigma)$ instead of $N(0, \sigma^2)$.

```
p = 12 # lag length
X = sm.tsa.tsatools.lagmat(y_full, maxlag=p, trim='both') # lag matrix

# Data for Stan
std_y = y_full.std()
stan_data = {
    'T': X.shape[0],
    'p': p,
    'y': y_full[p:],
    'X': X,
    'lambda': 0.2,
    'scale_sigma': std_y,
    'scale_c': 100*std_y,
}

# Compile and sample
model = CmdStanModel(stan_file='slides12_ar.stan')
fit = model.sample(data=stan_data, seed=31150, chains=4,
                   iter_warmup=1000, iter_sampling=1000, show_progress=True)

print(fit.summary())
```



- First Stan compiles the sampling program (only done once).
- Then runs 4 chains in parallel from randomly drawn initial values.
- For each chain: 1000 burn-in draws discarded, then last 1000 draws retained.
- Here sampling is very fast (5 seconds). Can take much longer in more complicated models.

```
            Mean      MCSE    StdDev       MAD           5%           50%  \
lp__   1010.280000  0.063370  2.759430  2.557710  1005.270000  1010.630000
c         0.089636  0.000391  0.025454  0.025671     0.047837     0.089861
a[1]      1.012370  0.000595  0.032229  0.032269     0.960301     1.011730
a[2]      0.172059  0.000791  0.042703  0.042244     0.099819     0.172488
a[3]     -0.045320  0.000651  0.037394  0.037117    -0.106699    -0.044841
[...]
a[12]     0.002350  0.000201  0.012718  0.012585    -0.018468     0.002253
sigma     0.192057  0.000074  0.004798  0.004618     0.184378     0.191976

            95%  ESS_bulk  ESS_tail  ESS_bulk/s     R_hat
lp__   1014.130000   1952.88   2720.81     169.344  1.000220
c         0.131474   4250.83   3084.38     368.612  1.000550
a[1]      1.065190   2965.11   2705.60     257.120  1.003520
a[2]      0.240726   2952.46   2658.34     256.024  1.000210
a[3]      0.015843   3348.71   2931.60     290.384  1.001960
[...]
a[12]     0.023426   4044.95   3335.40     350.758  1.000380
sigma     0.200040   4248.17   3160.73     368.381  1.001310
```

Note: parameters $a_4, a_5, \ldots, a_{11}$ omitted from output for legibility.

	Mean	MCSE	StdDev	MAD	5%	50%	\
lp__	1010.280000	0.063370	2.759430	2.557710	1005.270000	1010.630000	
c	0.089636	0.000391	0.025454	0.025671	0.047837	0.089861	
a[1]	1.012370	0.000595	0.032229	0.032269	0.960301	1.011730	
a[2]	0.172059	0.000791	0.042703	0.042244	0.099819	0.172488	
a[3]	-0.045320	0.000651	0.037394	0.037117	-0.106699	-0.044841	
[...]							
a[12]		00201	0.012718	0.012585	-0.018468	0.002253	
sigma		00074	0.004798	0.004618	0.184378	0.191976	

Posterior mean
$E[a_1 \mid \boldsymbol{Y}]$

	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	1014.130000	1952.88	2720.81	169.344	1.000220
c	0.131474	4250.83	3084.38	368.612	1.000550
a[1]	1.065190	2965.11	2705.60	257.120	1.003520
a[2]	0.240726	2952.46	2658.34	256.024	1.000210
a[3]	0.015843	3348.71	2931.60	290.384	1.001960
[...]					
a[12]	0.023426	4044.95	3335.40	350.758	1.000380
sigma	0.200040	4248.17	3160.73	368.381	1.001310

Note: parameters $a_4, a_5, \ldots, a_{11}$ omitted from output for legibility.

	Mean	MCSE	StdDev	MAD	5%	50%	\
lp__	1010.280000	0.063370	2.759430	2.557710	1005.270000	1010.630000	
c	0.089636	0.000391	0.025454	0.025671	0.047837	0.089861	
a[1]	1.012370	0.000595	0.032229	0.032269	0.960301	1.011730	
a[2]	0.172059	0.000791	0.042703	0.042244	0.099819	0.172488	
a[3]	-0.045320	0.000651	0.037394	0.037117	-0.106699	-0.044841	
[...]							
a[12]	0.002350	0.0		2585	-0.018468	0.002253	
sigma	0.192057	0.0		4618	0.184378	0.191976	

Posterior std. dev.
$\sqrt{\text{Var}(a_1 \mid \boldsymbol{Y})}$

	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	1014.130000	1952.88	2720.81	169.344	1.000220
c	0.131474	4250.83	3084.38	368.612	1.000550
a[1]	1.065190	2965.11	2705.60	257.120	1.003520
a[2]	0.240726	2952.46	2658.34	256.024	1.000210
a[3]	0.015843	3348.71	2931.60	290.384	1.001960
[...]					
a[12]	0.023426	4044.95	3335.40	350.758	1.000380
sigma	0.200040	4248.17	3160.73	368.381	1.001310

Note: parameters $a_4, a_5, \ldots, a_{11}$ omitted from output for legibility.

	Mean	MCSE	StdDev	MAD	5%	50%	\
lp__	1010.280000	0.063370	2.759430	2.557710	1005.270000	1010.630000	
c	0.089636	0.000391	0.025454	0.025671	0.047837	0.089861	
a[1]	1.012370	0.000595	0.032229	0.032269	0.960301	1.011730	
a[2]	0.172059	0.000791	0.042703	0.042244	0.099819	0.172488	
a[3]	-0.045320	0.000651	0.037394	0.037117	-0.106699	-0.044841	
[...]							
a[12]	0.002350	0.000201	0.012718	0.012585		0.002253	
sigma	0.192057	0.000074	0.004798	0.004618		0.191976	

Posterior 5th percentile of a_1

	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	1014.130000	1952.88	2720.81	169.344	1.000220
c	0.131474	4250.83	3084.38	368.612	1.000550
a[1]	1.065190	2965.11	2705.60	257.120	1.003520
a[2]	0.240726	2952.46	2658.34	256.024	1.000210
a[3]	0.015843	3348.71	2931.60	290.384	1.001960
[...]					
a[12]	0.023426	4044.95	3335.40	350.758	1.000380
sigma	0.200040	4248.17	3160.73	368.381	1.001310

Note: parameters $a_4, a_5, \ldots, a_{11}$ omitted from output for legibility.

	Mean	MCSE	StdDev	MAD	5%	50%	\
lp__	1010.280000	0.063370	2.759430	2.557710	1005.270000	1010.630000	
c	0.089636	0.000391	0.025454	0.025671	0.047837	0.089861	
a[1]	1.012370	0.000595	0.032229	0.032269	0.960301	1.011730	
a[2]	0.172059	0.000791	0.042703	0.042244	0.099819	0.172488	
a[3]	-0.045320	0.000651	0.037394	0.037117	-0.106699	-0.044841	
[...]							
a[12]	0.002350	0.000201	0.012718	0.012585	-0.01846		
sigma	0.192057	0.000074	0.004798	0.004618	0.18437		

	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	1014.130000	1952.88	2720.81	169.344	1.000220
c	0.131474	4250.83	3084.38	368.612	1.000550
a[1]	1.065190	2965.11	2705.60	257.120	1.003520
a[2]	0.240726	2952.46	2658.34	256.024	1.000210
a[3]	0.015843	3348.71	2931.60	290.384	1.001960
[...]					
a[12]	0.023426	4044.95	3335.40	350.758	1.000380
sigma	0.200040	4248.17	3160.73	368.381	1.001310

Posterior median
median($a_1 \mid \boldsymbol{Y}$)

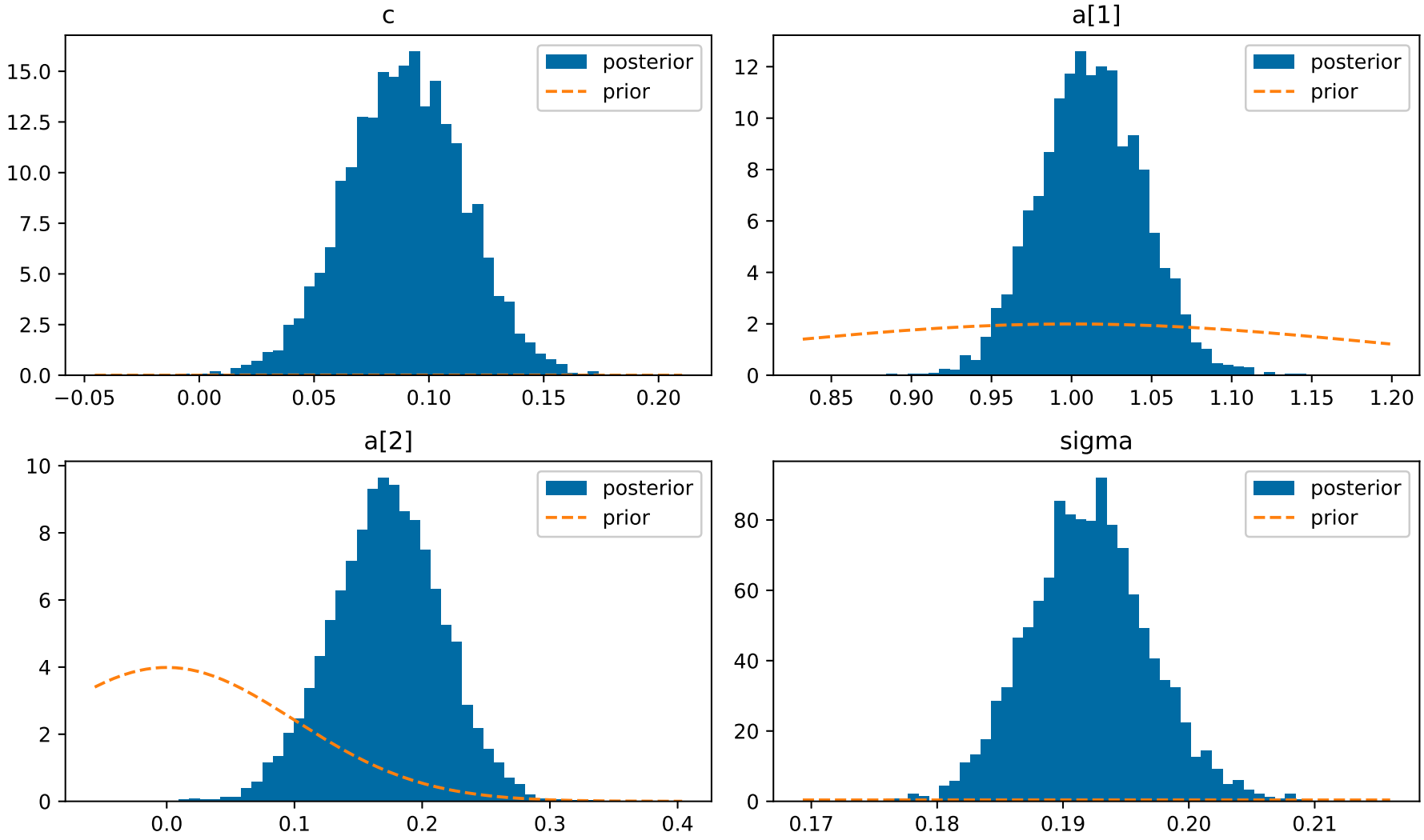
Note: parameters $a_4, a_5, \ldots, a_{11}$ omitted from output for legibility.

```
          Mean      MCSE    StdDev       MAD          5%          50%  \
lp__  1010.280000  0.063370  2.759430  2.557710  1005.270000  1010.630000
c        0.089636  0.000391  0.025454  0.025671     0.047837     0.089861
a[1]     1.012370  0.000595  0.032229  0.032269     0.960301     1.011730
a[2]     0.172059  0.000791  0.042703  0.042244     0.099819     0.172488
a[3]    -0.045320  0.000651  0.037394  0.037117    -0.106699    -0.044841
[...]
a[12]    0.002350  0.000201  0.012718  0.012585    -0.018468     0.002253
sigma    0.192057  0.000074  0.004798  0.004618     0.184378     0.191976

          95%  ESS_bulk  ESS_tail  ESS_bulk/s     R_hat
lp__  1014.130000   1952.88   2720.81     169.344  1.000220
c        0.131474   4250.83   3084.38     368.612  1.000550
a[1]     1.065190   2965.11   2705.60     257.120  1.003520
a[2]     0.240726   2952.46   2658.34     256.024  1.000210
a[3]     0.015843   3348.71   2931.60     290.384  1.001960
[...]
a[12]    0.044950   3335.40   3335.40     350.758  1.000380
sigma    0.048170   3160.73   3160.73     368.381  1.001310
```

Posterior 95th percentile of a_1

Note: parameters $a_4, a_5, \ldots, a_{11}$ omitted from output for legibility.



Note: prior densities for c and σ are so diffuse that they're barely visible.

```
# Posterior quantiles
print(np.quantile(draws['a[1]'].values, [0.025, 0.05, 0.95, 0.975]))
```

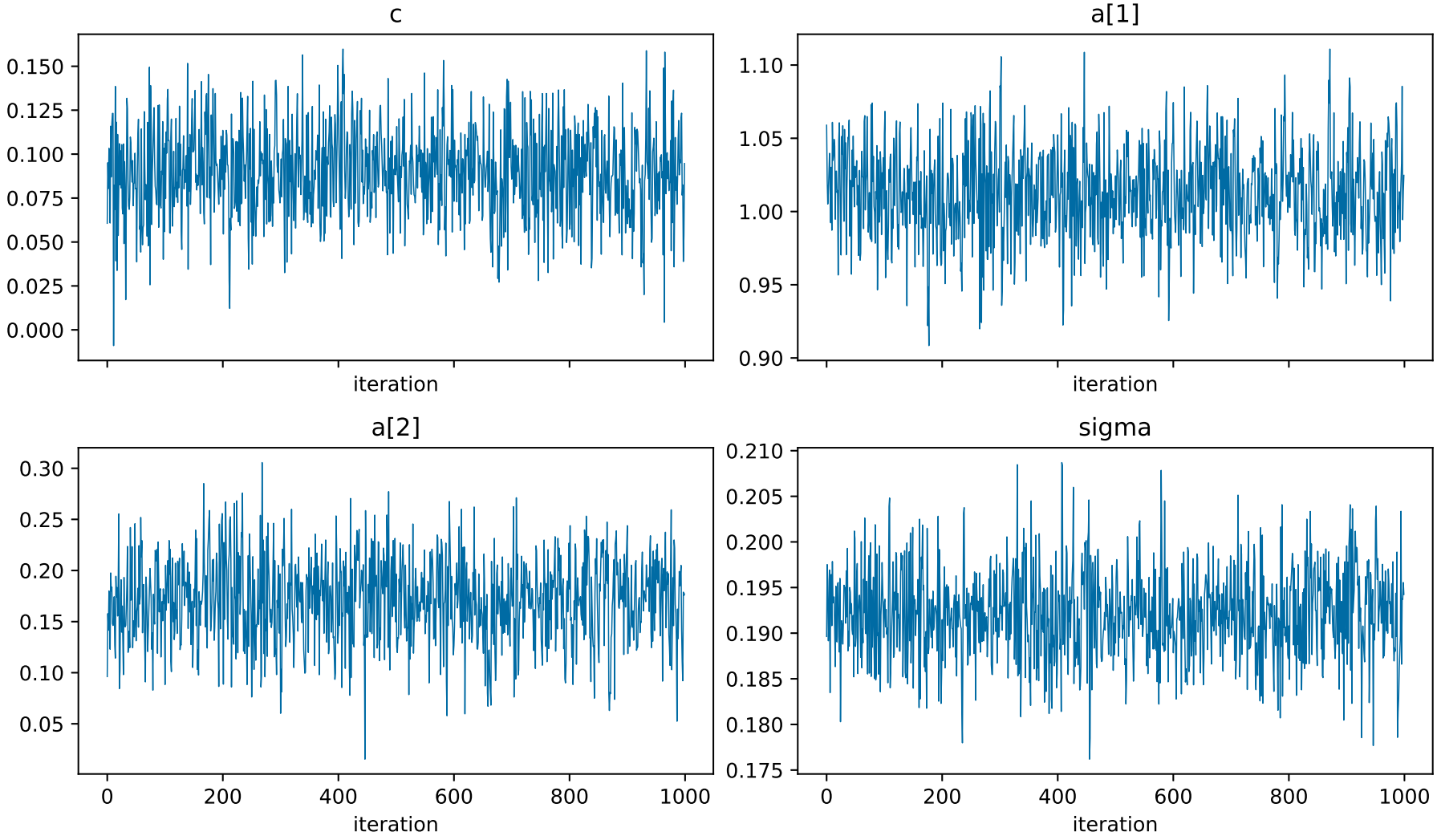
Result:

```
[0.95111886 0.96030143 1.06518581 1.07486885]
```

- 90% credible interval for a_1: $[0.960, 1.065]$
- 95% credible interval for a_1: $[0.951, 1.075]$

Convergence diagnostics

- It's important to check that the MCMC chain has reached a stationary distribution, after discarding the burn-in.
- Convergence may fail if the posterior density has multiple peaks or is very non-smooth in some directions.
- Plot the chains of each individual parameter (called a **trace plot**) and inspect whether the time series look (i) stationary and (ii) similar across parallel chains.



Plot of a single chain (out of 4 parallel ones) after burn-in

Convergence diagnostics (cont.)

- Stan's `R_hat` measures how different results are across parallel chains. Ideally, `R_hat` < 1.01.
- Each chain is serially correlated by design. If the dependence is very high, we need very many draws to accurately approximate posterior means, etc.
- Stan's `ESS_bulk` measures the "effective sample size", i.e., the number of draws that are approximately uncorrelated (adjusting for the long-run variance of the chain). Should be in the 1000s.

```
             Mean      MCSE    StdDev       MAD           5%           50%  \
lp__   1010.280000  0.063370  2.759430  2.557710  1005.270000  1010.630000
c         0.089636  0.000391  0.025454  0.025671     0.047837     0.089861
a[1]      1.012370  0.000595  0.032229  0.032269     0.960301     1.011730
a[2]      0.172059  0.000791  0.042703  0.042244     0.099819     0.172488
a[3]     -0.045320  0.000651  0.037394  0.037117    -0.106699    -0.044841
[...]
a[12]     0.002350  0.000201  0.012718  0.012585    -0.018468     0.002253
sigma     0.192057  0.000074  0.004798  0.004618     0.184378     0.191976

             95%  ESS_bulk  ESS_tail  ESS_bulk/s     R_hat
lp__   1014.130000   1952.88   2720.81     169.344  1.000220
c         0.131474   4250.83   3084.38     368.612  1.000550
a[1]      1.065190   2965.11   2705.60     257.120  1.003520
a[2]      0.240726   2952.46   2658.34     256.024  1.000210
a[3]      0.015843   3348.71   2931.60     290.384  1.001960
[...]
a[12]     0.023426   4044.95   3335.40     350.758  1.000380
sigma     0.200040   4248.17   3160.73     368.381  1.001310
```

	Mean	MCSE	StdDev	MAD	5%	50%	\
lp__	1010.280000	0.063370	2.759430	2.557710	1005.270000	1010.630000	
c	0.089636	0.000391	0.025454	0.025671	0.047837	0.089861	
a[1]	1.012370	0.000595	0.032229	0.032269	0.960301	1.011730	
a[2]	0.172059	0.000791	0.042703	0.042244	0.099819	0.172488	
a[3]	-0.045320	0.000651	0.037394	0.037117	-0.106699	-0.044841	
[...]							
a[12]	0.002350	0.000201	0.012718	0.012585	-0.018468	0.002253	
sigma	0.192057	0.000074	0.004798	0.004618	0.184378	0.191976	

	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	1014.130000	1952.88	2720.81	169.344	1.000220
c	0.131474	4250.83	3084.38	368.612	1.000550
a[1]	1.065190	2965.11	2705.60	257.120	1.003520
a[2]	0.240726	2952.46	2658.34	256.024	1.000210
a[3]	0.015843	3348.71	2931.60	290.384	1.001960
[...]					
a[12]	0.023426	4044.95	3335.40	350.758	1.000380
sigma	0.200040	4248.17	3160.73	368.381	1.001310

Good!

	Mean	MCSE	StdDev	MAD	5%	50%	\
lp__	1010.280000	0.063370	2.759430	2.557710	1005.270000	1010.630000	
c	0.089636	0.000391	0.025454	0.025671	0.047837	0.089861	
a[1]	1.012370	0.000595	0.032229	0.032269	0.960301	1.011730	
a[2]	0.172059	0.000791	0.042703	0.042244	0.099819	0.172488	
a[3]	-0.045320	0.000651	0.037394	0.037117	-0.106699	-0.044841	
[...]							
a[12]	0.002350	0.000201	0.012718	0.012585	-0.018468	0.002253	
sigma	0.192057	0.000074	0.004798	0.004618	0.184378	0.191976	

	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	1014.130000	1952.88	2720.81	169.344	1.000220
c	0.131474	4250.83	3084.38	368.612	1.000550
a[1]	1.065190	2965.11	2705.60	257.120	1.003520
a[2]	0.240726	2952.46	2658.34	256.024	1.000210
a[3]	0.015843	3348.71	0	290.384	1.001960
[...]					
a[12]	0.023426	4044.95	3335.40	350.758	1.000380
sigma	0.200040	4248.17	3160.73	368.381	1.001310

Good!

Convergence diagnostics (cont.)

- If you call `fit.diagnose()`, Stan performs some automatic convergence diagnostics (but you should also inspect yourself).
- How to think about and address convergence issues: `https://mc-stan.org/learn-stan/diagnostics-warnings.html`

What did we learn?

- Bayesian inference
  - Prior, likelihood, posterior
  - Credible interval
- MCMC
- Modeling and posterior sampling with Stan
  - Convergence diagnostics

Looking ahead

- Bayesian forecasting
- Bayesian (S)VARs
- Modeling time-varying shock volatility
- How to handle the massive Covid shock in economic data?